

Unit-III Array and Structure

Concept of Array

What is an Array?

- An **array** is a **linear data structure**.
- It is used to store **multiple values** in a **single variable**.
- All elements of an array are stored in **continuous memory locations**.
- An array can store **same type of data elements** (in C, C++, Java).

Why Array is Used?

When we need to store many values of **similar nature** (like marks of students), using separate variables is difficult. Array makes it easy and organized.

Syntax of Array:

```
data_type array_name[array_size];
```

Declaration and Initialization: `data_type array_name[array_size] = {value1, value2, value3};`

Example: `int marks[5] = {60, 70, 80, 90, 100};`

3.1 characteristics of arrays

1. Collection of Elements

An array stores **multiple values** under **one single name**.

2. Same Data Type

All elements of an array are of the **same data type** (int, float, char, etc.).

3. Fixed Size

The size of an array is **fixed** and must be decided **before use**.

4. Stored in Contiguous Memory

Array elements are stored in **continuous (adjacent) memory locations**.

5.Indexed Structure

Each element is accessed using an **index number** starting from **0**.

6.Easy Access of Elements

Any element can be accessed **directly** using its index.

7.Single or Multi-Dimensional

Arrays can be **one-dimensional**, **two-dimensional**, or **multi-dimensional**.

8.Efficient for Similar Data

Arrays are useful when we need to store and process **large amounts of similar data**.

Types of Array

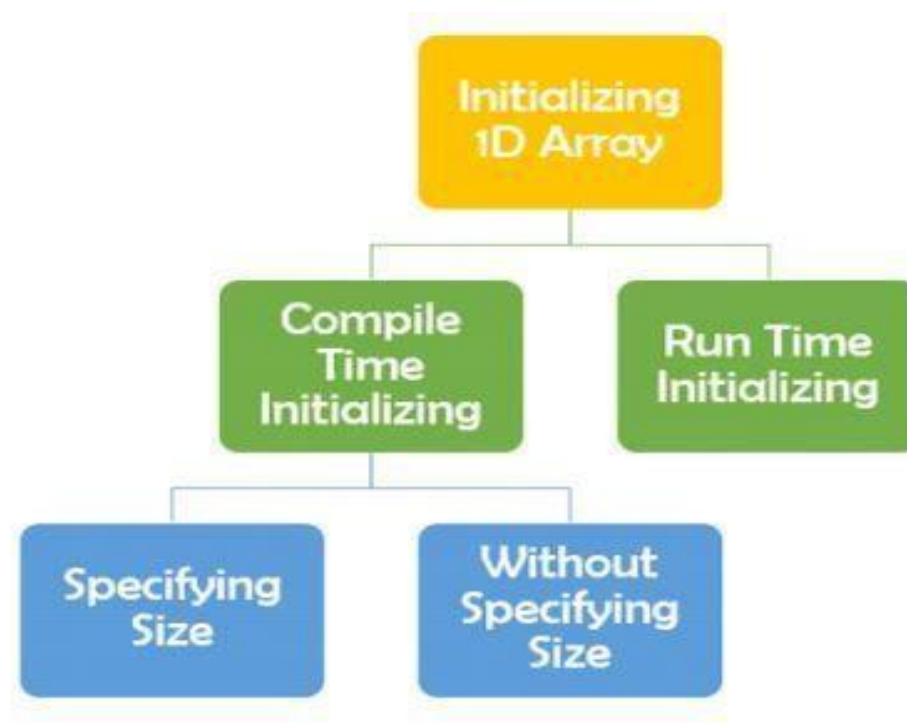
- 1.Single Dimensional Array / One Dimensional Array(1D)
- 2.Two dimensional arrays(2D)
- 3.Multi Dimensional Array

Single / One Dimensional Array:

1. Single or One Dimensional array is used to represent and store data in a linear form.
2. Array having only one subscript variable is called One-Dimensional array
3. It is also called as Single Dimensional Array or Linear Array

Single Dimensional Array (1D) Declaration and initialization:

1. **Syntax for declaration:** <data_type><array_name> [size];
2. **Examples for declaration:** int iarr[3]; char carr[20]; float farr[3];
3. **Syntax for initialization:** [size] = {val1, val2, ..., valn};
4. **Examples for initialization:** int iarr[3] = {2, 3, 4}; char carr[20] = "program"; float farr[3] = {12.5, 13.5, 14.5};
5. **Different Methods of Initializing 1-D Array** Whenever we declare an array, we initialize that array directly at compile time. Initializing 1-D Array is called as compiler time initialization if and only if we assign certain set of values to array element before executing program. i.e. at compilation time.



Here we are learning the different ways of compile time initialization of an array.

Ways of Array Initializing 1-D Array:

1. Size is Specified Directly
2. Size is Specified Indirectly

Example Program: Accessing 1-D Array

```
#include<stdio.h>
#include<conio.h>
void main()
{
int arr[] = {51, 32, 43, 24, 5, 26};
int i;
    for(i = 0; i < 6; i++)
    {
        printf("\nElement at arr[%d] is %d", i, arr[i]);
    }
    getch();
}
```

Consider the array:

```
int arr[] = {51, 32, 43, 24, 5, 26};
```

Important Points: Arr → Base address
of array arr[i] → Element stored at
position I

Internally: arr[i] =

*(arr + i)

Example Program: Different Ways to Access Same Element

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int arr[] = {51, 32, 43, 24, 5, 26};
```

```
int i;
```

```
for(i = 0; i < 6; i++)
```

```
{
```

```
printf("\n%d %d %d %d", arr[i], *(arr+i), *(i+arr), i[arr]);
```

```
}
```

```
getch();
```

```
}
```

Two Dimensional Arrays

1. Two Dimensional Array requires Two Subscript Variables
2. Two Dimensional Array stores the values in the form of matrix.
3. One Subscript Variable denotes the “Row” of a matrix.
4. Another Subscript Variable denotes the “Column” of a matrix.

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Declaration and use of 2D Arrays:

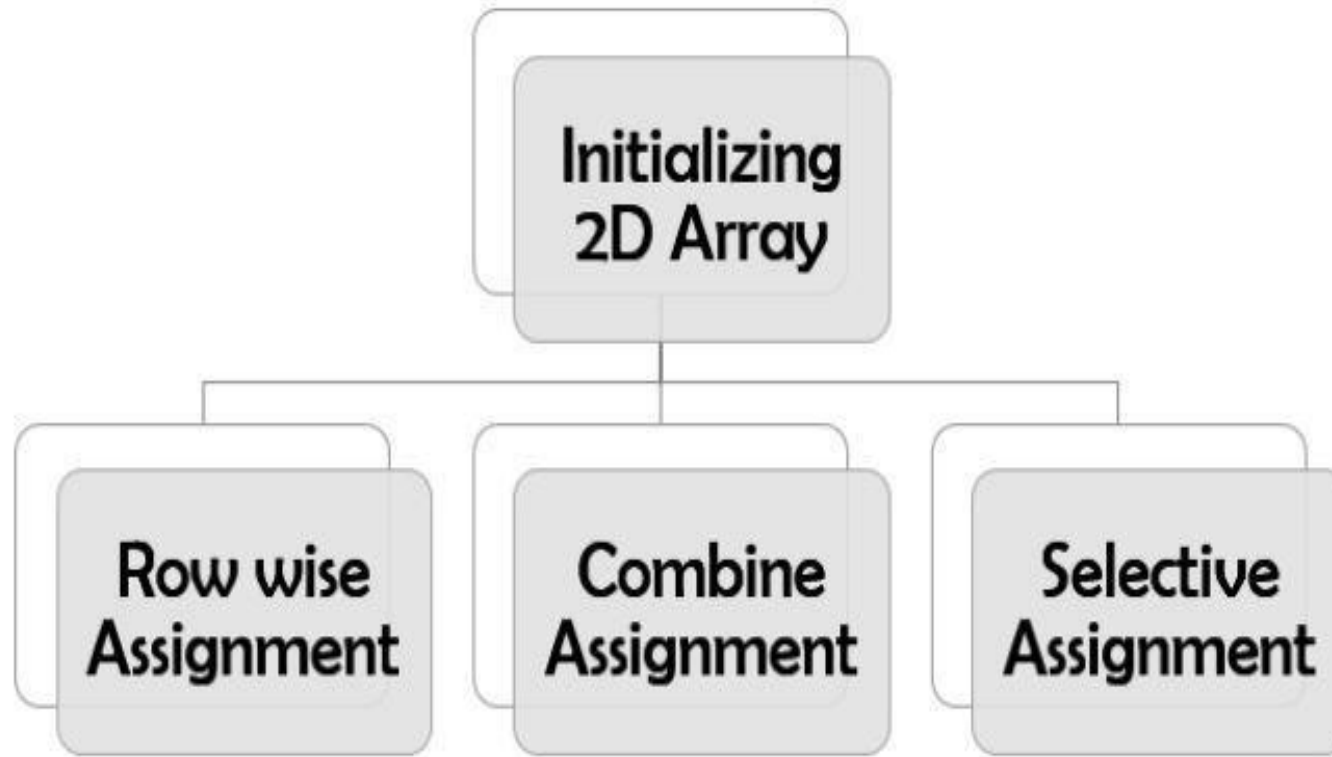
```
int a[3][4];
```

```
for(i=0;i<row,i++)
```

```
for(j=0;j<col,j++)
```

```
{  
printf("%d",a[i][j]);  
}
```

Initializing 2D Array



Method 1:

Initializing all Elements row wise For initializing 2D Array we need to assign values to each element of an array using the below **syntax**.

```
int a[3][2] = { {1, 4}, {5, 2}, {6, 5} };
```

Example Program: Two Dimensional Array

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i, j;
    int a[3][2] = { {1, 4}, {5, 2}, {6, 5} };
    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 2; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
    getch();
}
```

Output:

1 4

5 2

6 5

Accessing 2-D Array Elements

Key Points

1. A **2-D array** requires **two subscript variables**.
2. $i \rightarrow$ refers to **row number**.
3. $j \rightarrow$ refers to **column number**.
4. $a[1][0] \rightarrow$ element of **first row** and **zeroth column**.

Example Program: Accept & Print 3×3 Matrix

```
#include<stdio.h>
#include<conio.h>

void main()
{

    int i, j, a[3][3];
```

```
/* Accepting elements */  
for(i = 0; i < 3; i++)  
{  
    for(j = 0; j < 3;  
        j++)  
    {  
        printf("\nEnter the a[%d][%d] = ", i,j);  
        scanf("%d", &a[i][j]);  
    }  
}
```

```
/* Printing elements */
```

```
printf("\nMatrix elements are:\n");  
for(i = 0; i <3; i++)  
{  
    for(j = 0; j < 3; j++)  
    {  
        printf("%d\t", a[i][j]);  
    }  
}
```

```
        printf("\n");  
    }  
  
    getch();  
}
```

Sample Output:

Matrix elements are:

```
1  2  3  
4  5  6  
7  8  9
```

Example programs for practice:

1. C Program for addition of two matrices
2. C Program to find inverse of 3 X # Matrix
3. C Program to Multiply two 3 X 3 Matrices
4. C Program to check whether matrix is magic square or not?

Program: Addition of Two Matrices

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

void main()
{
    int i, j;
    int mat1[10][10], mat2[10][10], mat3[10][10];

    int row, col;

    clrscr();

    printf("Enter number of rows: ");
    scanf("%d", &row);

    printf("Enter number of columns: ");
    scanf("%d", &col);
    /* Accept elements of Matrix 1 */
    printf("\nEnter elements of Matrix 1:\n");
    for(i = 0; i < row; i++)
```

```
{  
for(j = 0; j < col; j++)  
{  
    printf("Enter mat1[%d][%d]: ", i, j);  
    scanf("%d", &mat1[i][j]);  
}  
}
```

```
/* Accept elements of Matrix 2 */  
printf("\nEnter elements of Matrix 2:\n");  
for(i = 0; i < row; i++)  
{  
    for(j = 0; j < col; j++)  
    {  
        printf("Enter mat2[%d][%d]: ", i, j);  
        scanf("%d", &mat2[i][j]);  
    }  
}
```

```
for(i = 0; i < row; i++)
```

```
{  
for(j = 0; j < col; j++)  
    {  
        mat3[i][j] = mat1[i][j] + mat2[i][j];  
    }  
}  
printf("\nAddition of two matrices:\n");  
    for(i = 0; i < row; i++)  
    {  
for(j = 0; j < col; j++)  
    {  
        printf("%d\t", mat3[i][j]);  
    }  
        printf("\n");  
    }  
    getch()  
;  
}
```

Concept of Multi-Dimensional Arrays

Definition

- A **multi-dimensional array** is an array that has **more than one dimension**.
- It is used to store data in **row and column form**.

Two-Dimensional Array

- A **2-D array** is also called a **matrix**.
- It stores data in the form of **rows and columns**.

3.2 Array declaration and Initialization

1. Array Declaration

Array declaration means **defining an array name, data type, and size**. It tells the compiler how much memory to reserve.

Syntax:

```
data_type array_name[size];
```

Examples:

```
int a[5];  
float marks[10]; char  
name[20];
```

2. One-Dimensional Array Declaration Used to store a list of similar elements. `int numbers[5];`

3. Array Initialization

Array initialization means **assigning values to array elements**.

Method 1: Initialization at Declaration Time

```
int a[5] = {10, 20, 30, 40, 50};
```

Method 2:

Initialization Without Specifying Size Compiler automatically decides the size. `int a[] = {1, 2, 3, 4};`

Method 3:

Partial Initialization

Remaining elements are set to 0 automatically.

```
int a[5] = {10, 20};
```

Stored as:{10, 20, 0, 0, 0}

Method 4:Initialization Using Index

```
int a[5]; a[0] = 5; a[1] = 10; a[2] = 15;
```

4. Two-Dimensional Array DeclarationUsed to store data in **rows and columns**.

Syntax:

```
data_type array_name[row][column];
```

Example:

```
int matrix[3][2];
```

Two-Dimensional Array Initialization

```
int matrix[3][2] = { {1, 2}, {3, 4}, {5, 6} };
```

Initialization Without Inner Braces

```
int matrix[2][3] = {1, 2, 3, 4, 5, 6};
```

Accessing Array Elements

Array elements are accessed using **index numbers**.

Index always starts from **0**.

```
a[0] = 10; matrix[1][2] = 5;
```

3.3 Operations on Arrays.

Types of Operations on Array

1. Traversal

1. Visiting and accessing **each element** of the array one by one.
2. Commonly done using loops.

2. Insertion

1. Adding a **new element** at a particular position in the array.
2. Elements are shifted to create space.

3. **Deletion**

1. Removing an element from a specific position.
2. Remaining elements are shifted to fill the gap.

4. **Searching**

1. Finding the **location of an element** in the array.
2. Can be **Linear Search** or **Binary Search**.

1) **Traversal**

Traversal is the operation of accessing and visiting each element of the array sequentially.

Write a C program to traverse the array display all the elements.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[5], i;
    clrscr();
    printf("Enter 5
elements:\n");
```

```
for(i = 0; i < 5; i++)
    scanf("%d", &a[i]);
printf("Array elements are:\n");
    for(i = 0; i < 5; i++)
        printf("%d\n", a[i]);

getch();
}
```

2. Sum and Average of Array Elements

```
#include<stdio.h>
#include<conio.h>
```

```
void main()
{
    int a[5], i, sum=0;
    float avg;
    clrscr();
    for(i=0;i<5;i++)
    {
        scanf("%d",&a[i]);
        sum = sum + a[i];
    }
}
```

```
avg = sum / 5.0;
printf("Sum = %d",sum);
printf("\nAverage = %f",avg);
getch();
}
```

3. Largest Element in Array

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[5], i, max;
clrscr();
for(i=0;i<5;i++)
scanf("%d",&a[i]);

max = a[0];
for(i=1;i<5;i++)
{
if(a[i] > max)
max = a[i];
}
```

```
}  
printf("Largest element = %d",max);  
getch();  
}
```

2) Insertion

Insertion is the operation of adding a new element at a specified position in an array.

Write a C program to insert the element 25 at position 3 in the array 10 20 30 40 50

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    int  
    a[10]={10,20,30,40,50};  
    int i, n=5, pos=3, ele=25;  
    clrscr();  
  
    for(i=n;i>=pos;i--)  
        a[i]=a[i-1];  
  
    a[pos-1]=ele;  
    n++;
```

```
    for(i=0;i<n;i++)  
        printf("%d ",a[i]);  
  
    getch();  
}
```

3) Deletion :Deletion is the operation of removing an element from a specified position in an array.

Write a C program to delete the element at position 4 from the array 10 20 30 40 50

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    int  
  
    a[10]={10,20,30,40,50};  
  
    int i, n=5, pos=4;  
  
    clrscr();  
  
    for(i=pos-1;i<n-  
    1;i++) a[i]=a[i+1];  
  
    n--;
```

```
    for(i=0;i<n;i++)  
        printf("%d ",a[i]);  
  
    getch();  
}
```

4) Searching (Linear Search)

Searching is the operation of finding the location of an element in an array.

Write a C program to search the element 30 in the array 10 20 30 40 50

```
#include<stdio.h>  
  
#include<conio.>  
  
void main()  
{  
    Int a[5]={10,20,30,40,50};  
    int i,  
    key=30;  
    clrscr();  
  
    for(i=0;i<5;i++)
```

```
{  
if(a[i]==key)  
    {  
        printf("Element found at  
        position %d",i+1); getch();  
        return;  
    }  
}  
  
printf("Element not found");  
    getch();  
}
```

3.4: Character and String Input / Output & String Operations

In C programming, data can be handled either as a single character or as a group of characters called a string.

1. CHARACTER INPUT AND OUTPUT

A character is a single letter, digit, or special symbol.

In C language, character data type is represented using 'char'.

Character input/output functions are used to read or display one character at a time. These functions are commonly used in Turbo C programming.

Important functions:

1) getch() – Reads a character from keyboard without displaying it on screen.

2) putchar() – Displays a character on screen.

Header file required: conio.h

PROGRAM: Character Input and Output

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    char ch;
```

```
    clrscr();
```

```
    printf("Enter a character: ");
```

```
    ch = getch();
```

```
    printf("\nEntered character is: ");
```

```
    putchar(ch);  
    getch();  
}
```

Explanation:

- char ch declares a character variable. - getch() accepts one character. - putchar(ch) prints the character

2. STRING INPUT AND OUTPUT

A string is a collection of characters stored in a character array. Every string ends with a null character '\0'.

Example:char name[20];

String input/output functions:

- 1) **gets()** – Reads a string from keyboard.
- 2) **puts()** – Displays a string.

PROGRAM: String Input and Output

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    char name[20];
```

```
    clrscr();
```

```
    printf("Enter name: "); gets(name);
```

```
    printf("Entered name is: "); puts(name);
```

```
    getch();
```

```
}
```

3. STRING LENGTH

- **strlen()** function is used to find the length of a string. It does not count the null character.

Header file: string.h

PROGRAM: String Length

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char str[20]; int len;
    clrscr();
    gets(str);
    len = strlen(str);
    printf("Length = %d", len);

    getch();
}
```

4. STRING COPY

- **strcpy()** function copies one string into another.

Syntax: strcpy(destination,
source);

PROGRAM: String Copy

```
#include <stdio.h>
#include <conio.h>

#include <string.h>

void main()
{
    char s1[20], s2[20];
    clrscr();
    gets(s1);

    strcpy(s2, s1);
    puts(s2);

    getch();
}
```

5. STRING COMPARE

- **strcmp()** compares two strings.
If strings are equal, it returns 0.

PROGRAM: String Compare

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char s1[20], s2[20];

    clrscr();
    gets(s1);
    gets(s2);

    if(strcmp(s1,s2) == 0)

        printf("Strings are equal");

    else printf("Strings are not equal");

    getch();
}
```

6. STRING CONCATENATION

- **strcat()** joins two strings.

PROGRAM: String Concatenation

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char s1[20], s2[20];
    clrscr();
    gets(s1);
    gets(s2);

    strcat(s1, s2);
    puts(s1);

    getch();
}
```

7. STRING REVERSE

- Reversing means displaying string in reverse order.
strrev() is used in C. **PROGRAM: Reverse String**

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <string.h>
```

```
void main()
```

```
{
```

```
char str[20];
```

```
clrscr();
```

```
gets(str);
```

```
strrev(str);
```

```
puts(str);
```

```
getch();
```

```
}
```

8. PALINDROME STRING

- A palindrome string reads the same forward and backward.

Examples: MADAM, LEVEL

PROGRAM: Palindrome String

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char str[20], temp[20];
    clrscr();

    gets(str);
    strcpy(temp, str);
    strrev(str);

    if(strcmp(str, temp) == 0)
        printf("Palindrome String");
    else printf("Not a Palindrome");
    getch();
}
```

3.5 Introduction and Features of Structures

1. In C language, a **structure** is a user-defined data type.
2. It is used to **store different types of data items together** under a single name.
3. Structure allows grouping of **related data** of different data types.
4. Unlike arrays, structures can store **int, float, char, double, etc. in one variable**.
5. Each data item inside a structure is called a **structure member**.
6. Structures are useful when a program needs to handle **complex data records**.
7. Structures help in representing **real-world entities** like student, employee, book, etc.

Features of Structures

- 1. User-defined data type**
Structure is created by the programmer according to program requirement.
- 2. Stores different data types**
It can store variables of different data types under one name.
- 3. Groups related data**
All related information can be kept together, making the program easy to understand.
- 4. Each member has a unique name**
Every structure member is accessed using its own name.
- 5. Access using dot (.) operator**
Structure members are accessed using the dot operator.
- 6. Supports array of structures**
Multiple structure variables can be stored using arrays.
- 7. Can be passed to functions**
Structure variables can be passed as arguments to functions.
- 8. Improves program readability**
Using structures makes programs more organized and readable.
- 9. Supports nesting**
One structure can be declared inside another structure.
- 10. Used for record management**
Widely used in applications like databases, files, and data records.

Difference between Array and Structure

Array

- Array is a collection of **same data type** elements.
- All elements in an array must be of **same type** (int, float, char, etc.).
- Array elements are accessed using **index number**.
- Array uses **square brackets []** for access.
- Array is mainly used to store **list of similar items**.
- Example: marks of students
- Array size is fixed at declaration.

Defining a Structure(Definition)

Structure

- Structure is a collection of **different data type** elements.
- Members in a structure can be **int, float, char, double, etc.**
- Structure members are accessed using **member name**.
- Structure uses **dot (.) operator** for access.
- Structure is used to store **record of related data**.
- Example: student record (roll no, name, marks)
- Structure can store different types of values in one variable.

- Defining a structure means **creating a new user-defined data type** in C.
- It is done using the struct **keyword**.
- While defining a structure, **memory is not allocated**.
- It only **specifies the structure name and its members**.

Example: Define a structure to store student information.

```
struct student
{
    int roll_no; char
    name[20];
    float marks;
};
```

Initialization of Structure

1. Initialization of structure means **assigning values to structure members**.
2. It is done **at the time of declaring the structure variable**.
3. Values are assigned **in the same order** as structure members.
4. Structure members can also be initialized **individually using dot (.) operator**.

Method 1: Initialization at Declaration struct student

```
{  
    int roll_no;  
    char name[20];  
    float marks;  
};  
struct student s1 = {1, "Amit", 78.5};
```

Method 2: Initialization using Dot Operator

```
struct student s2;
```

```
s2.roll_no = 2;  
strcpy(s2.name, "Riya");  
s2.marks = 85.0;
```

Array of Structure

Definition

Array of structure is a collection of structure variables of the same type stored using a single array name.

Syntax

```
struct structure_name array_name[size];
```

Example: Store details of 3 students using array of structure.

```
struct student
{
    int roll_no;
    float marks;
};
```

```
struct student s[3];
```

Accessing Array of Structure

Definition

Accessing array of structure means accessing the members of each structure element using array index and dot (.) operator.

Syntax

```
array_name[index].member_name;
```

Write a C program to declare a structure student having data members roll number and percentage. Accept data for five students and display the roll number of the student having highest percentage.

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
int roll_no;
```

```
float percentage;
```

```
};
```

```
void main()
```

```
{
```

```
struct students[5];
```

```
int i,
```

```
max_index = 0;
```

```
clrscr();
```

```
/* Accept data */
```

```
for(i = 0; i < 5; i++)
```

```
{
```

```
    printf("\nEnter roll number of
```

```
    student %d: ", i+1);
```

```
    scanf("%d", &s[i].roll_no);
```

```
    printf("Enter percentage of
```

```
    student %d: ", i+1);
```

```
    scanf("%f", &s[i].percentage);
```

```
}
```

```
/* Find highest percentage */
```

```
for(i = 1; i < 5; i++)
```

```
{
```

```
    if(s[i].percentage > s[max_index].percentage)
```

```
    {
```

```
        max_index = i;
```

```
    }
```

```
}
```

```
/* Display result */

printf("\nStudent with highest percentage:");
printf("\nRoll Number = %d", s[max_index].roll_no);
printf("\nPercentage = %.2f", s[max_index].percentage);

getch();
}
```

Write a C program to declare a structure student having data members name, roll number and city. Accept data for one student and display the accepted result.

```
#include<stdio.h>
#include<conio.h>
struct student
{
    char name[20];
    int roll_no;
    char city[20]; };
}
```

```
void main()
{
    struct student s;

    clrscr();

    printf("Enter student name: ");
    scanf("%s", s.name);

    printf("Enter roll number: ");
    scanf("%d", &s.roll_no);

    printf("Enter city: ");
    scanf("%s", s.city);

    printf("\n--- Student Details ---");

    printf("\nName   : %s", s.name);

    printf("\nRoll Number: %d",s.roll_no);
```

```
printf("\nCity : %s", s.city);
```

```
    getch();  
}
```

3.6 typedef and Enumerated Data Type

A) typedef

- Typedef is used to **create a new name (alias) for an existing data type**.
- It **does not create a new data type**, only gives another name.
- It helps to **simplify complex declarations**.
- It makes the program **easy to read and understand**.
- typedef is used to give a new name to an existing data type.

Syntax:

```
typedef    existing_data_type  
new_name;
```

Example: typedef int number;

```
void main()
{
    number a = 10;
    printf("%d", a);
}
```

B) Enumerated Data Type (enum)

- Enum is a **user-defined data type**.
- It is used to **define a set of named integer constants**.
- The values of enum constants are **integers starting from 0 by default**.
- Enum improves **program readability**.
- It is mainly used where **fixed options** are available.

Syntax :

```
enum enum_name {constant1, constant2, constant3};
```

Example :

```
#include<stdio.h>
enum day {SUN, MON, TUE, WED}; void
main()
{
    enum day d; d =
    MON;
    printf("%d", d);
}
```

Program Print All Months Using enum and for Loop:

```
#include<stdio.h>
#include<conio.h>

enum month {JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC};

void main()
{
```

```
int i;
```

```
clrscr();
```

```
printf("Months enum values:\n");
```

```
for(i = JAN; i <= DEC; i++)  
{
```

```
printf("%d\n", i);
```

```
}
```

```
getch();
```

```
}
```